

OOT_Chapter9

OOT Chapter 9 领域模型开卷速查材料

1. 领域模型是什么

1.1 定义

领域模型 Domain Model 是对领域内概念类的可视化表示。

它也可以叫：

- 概念模型
- 领域对象模型
- 分析对象模型
- 可视化字典

领域模型通常用 UML 类图表示，但它不是最终代码类图，而是业务概念图。

1.2 领域模型展示什么

领域模型主要展示三类内容：

1. 领域对象 / 概念类
2. 概念类之间的关联
3. 概念类的属性

考试截图里说的：

概念类、概念类之间的关系、基本属性

本质上就是 Chapter 9 的这三项。

1.3 为什么要做领域模型

领域模型的作用：

1. 提高抽象层次；
2. 减少现实业务和软件模型之间的表达差距；
3. 帮助理解领域问题；

4. 为后续系统顺序图、操作契约、设计类图提供基础。

考试答题时可以写：

领域模型用于表达业务领域中的重要概念、概念之间的关系以及概念的基本属性，帮助我们理解需求，而不是描述具体软件实现。

2. 领域模型不是什么

这是判断题和简答题的高频点。

2.1 领域模型不是软件对象模型

领域模型中不要出现：

- 窗口
- 页面
- 按钮
- 控制器
- 数据表
- 数据库访问类
- DAO
- Service 方法
- 算法
- 函数
- 职责和方法

错误示例：

报修页面

提交按钮

RepairController

RepairService

RepairDatabase

sendNotification()

calculateTotalCost()

正确示例：

报修单

调度记录

活动记录

维修工人
费用单
支付记录

考试里如果看到“领域模型中应包含方法和职责”，一般是错的。

2.2 领域模型不是数据模型

领域模型关注：

业务中有哪些概念。

数据模型关注：

这些概念如何被持久化。

所以领域模型不等于数据库表设计。

例如：

领域模型关心：报修单、调度记录、活动记录、维修工人。

数据模型关心：RepairOrder 表、DispatchRecord 表、外键、索引、字段类型。

考试答法：

领域模型不是数据库表设计。领域模型关注业务概念及其关系，数据模型关注持久化结构。二者可以有关联，但不能混淆。

3. 如何创建领域模型

Chapter 9 给出的基本步骤是：

1. 寻找概念类；
2. 将概念类绘制为 UML 类图中的类；
3. 添加关联和属性。

考试时可以扩展为 6 步：

读需求 → 圈名词 → 筛概念类 → 加属性 → 建关联 → 标多重性

更完整的答题流程：

1. 从需求中识别名词和名词短语；
2. 判断哪些名词是需要记录、查询、统计的业务概念；

- 3. 把这些概念画成类；
- 4. 从需求中的“记录内容”提取属性；
- 5. 根据业务规则建立关联；
- 6. 标出多重性，如 1、0..1、0..、1..；
- 7. 对复杂约束用 note 补充。

4. 什么是概念类

一个概念类可以从三个角度理解：

角度	含义	例子
符号	表示概念类的词语或图形	报修单
内涵	概念类的定义 / 意义	记录业主一次报修请求的业务对象
外延	概念类的一组示例	某业主 6 月 11 日提交的停电报修单

考试中不需要专门写“符号、内涵、外延”，但它能帮助你判断一个名词是不是概念类：

如果一个名词能被定义，并且现实中有多个实例，通常更可能是概念类。

5. 如何寻找概念类

Chapter 9 给出三种策略：

- 1. 重用和修改已有模型；
- 2. 使用分类列表；
- 3. 识别名词短语。

5.1 策略一：重用和修改已有模型

如果题目基于 Lab2 扩展，那么你应优先保留 Lab2 中已有的主干模型，再根据新需求扩展。

物业报修主干可以先想成：

业主 → 报修单 → 调度记录 → 活动记录
维修工人 → 调度记录
报修单 → 评价单
报修单 → 投诉单
维修工人 ↔ 故障类别

如果题目新增“巡检”，就在主干旁边加：

- 巡检任务
- 巡检记录
- 设施

如果题目新增“材料成本”，加：

- 维修材料
- 材料使用记录

如果题目新增“费用结算”，加：

- 费用单
- 支付记录

5.2 策略二：使用分类列表

课件给出的分类列表非常适合开卷时查。

分类	POS 例子	物业报修可类比
业务交易	Sale / Payment	报修单、费用单、支付记录
交易项目	SalesLineItem	活动记录、派工明细、材料使用记录、巡检记录
与交易项目相关的产品或服务	Item / Flight / Seat	维修材料、设施、维修服务、故障类别
交易记录发生何处	Register	房屋、小区区域、设施位置
与交易相关的人或组织角色	Cashier / Store	业主、调度员、维修工人、物业经理
交易 / 活动地点	Store	房屋、小区区域、楼栋、单元
重要事件	Sale / Payment	报修、调度、维修活动、巡检、投诉、评价、支付
物理对象	Register	设施、维修材料、工具、设备
事物描述	ProductDescription	故障类别、材料类别、设施类型
类别	ProductCatalog	故障类别目录、材料类别、设施类型
事物容器	Store / Bin	仓库、库位、小区、楼栋
其他协作系统	CreditAuthorizationSystem	支付系统、短信系统、库存系统

分类	POS 例子	物业报修可类比
金融 / 工作 / 合约记录	Receipt / MaintenanceLog	费用单、支付记录、活动记录、绩效考核
金融手段	Cash / Check	支付方式
执行工作所需进度表、手册、文档	RepairSchedule	排班记录、巡检计划、维修规程

使用方法：

如果你想不到概念类，就把需求中的名词放到这个表里对照，看它属于哪类业务对象。

5.3 策略三：识别名词短语

从需求文本中圈出名词和名词短语，把它们作为候选概念类或属性。

例如需求：

业主通过电话报修，调度员录入报修信息，维修工人处理维修任务，并填写维修记录。任务完成后，业主进行评价。

候选名词：

业主、电话、调度员、报修信息、维修工人、维修任务、维修记录、任务、评价

筛选后：

候选名词	是否建类	理由
业主	建类	业务参与者，需要关联报修单
电话	一般不建类	通常只是报修来源属性
调度员	建类	业务角色，需要录入、调度
报修信息	建成报修单	需要长期保存和跟踪
维修工人	建类	业务角色，需要派工和统计
维修记录	建成活动记录	用于记录过程和工时
评价	建成评价单	有评分和时间，需保存
任务	看情况	如果已用报修单 / 调度记录表达，不必重复建类

6. 如何判断一个名词是否应该建成概念类

6.1 建类的常见条件

一个对象满足下面任意几项，就很适合建成概念类：

- 1. 需要长期保存；
- 2. 需要被查询；
- 3. 需要参与统计；
- 4. 有自己的编号、时间、状态、内容等属性；
- 5. 与多个其他对象有关联；
- 6. 生命周期独立，例如创建、完成、关闭；
- 7. 需求中反复出现；
- 8. 关系本身有属性，需要抽成中间类。

6.2 通常不建类的东西

下面这些通常不要直接建类：

类型	例子	原因
动作	提交、选择、点击、计算、通知	通常是行为，不是静态概念
界面元素	页面、按钮、窗口、表单	领域模型不是 UI 模型
技术实现	数据库、表、控制器、DAO	领域模型不是软件实现模型
简单属性	电话、地址、状态、备注	通常作为类的属性
派生结果	总工时、总材料费、是否空闲	通常可由其他对象计算得到

例外：

如果系统要求保存某个动作的记录，它就可以建成类。

通知 → 通常不是类

通知记录 → 如果要保存通知时间、方式、状态，就可以是类

提交 → 通常不是类

提交记录 → 如果要审计提交人、提交时间，就可以是类

统计 → 通常不是类

统计报表 / 绩效考核 → 如果要保存结果，就可以是类

7. 属性怎么找

属性通常来自需求里的“记录内容”。

常见属性类型：

属性类型	例子
编号	repairId、dispatchId、employeeId
时间	requestTime、dispatchTime、startTime、endTime、finishTime
文本	description、content、remark、suggestion
状态	status
数量	quantity、stockQuantity
金额	unitPrice、materialCost、totalAmount
评分	responseTimeliness、serviceAttitude、satisfactionResult
类型	source、paymentMethod、facilityType

7.1 属性和概念类的区别

判断口诀：

简单值作属性，复杂对象建类。

例如：

信息	建模方式	理由
报修来源：电话 / 微信	报修单.source	只是简单枚举值
报修人姓名	业主.name	属于业主属性
维修材料名称	维修材料.name	如果要维护库存和单价，材料应建类
材料使用数量	材料使用记录.quantity	数量属于“某活动使用某材料”的关系
支付方式	支付记录.paymentMethod	简单属性
支付记录	建类	有支付时间、金额、状态、方式
总工时	派生属性或不写	可由活动记录 startTime/endTime 计算
是否空闲	派生属性或 note	可由排班、不可用记录、活动调度推导

8. 关联怎么建

8.1 关联是什么

关联是概念类之间**有意义、值得关注的连接**。

课件强调两点：

- 1. 不要画大量无关关联，否则图会混乱；
- 2. 不要从软件实现角度画关联，画关联是为了表达对重要业务关系的理解。

8.2 什么时候要建关联

当需求中出现这些表达时，通常要建关联：

需求表达	关联
A 发起 B	A -- B
A 创建 B	A -- B
A 包含 B	A *-- B 或 A -- B
A 对应 B	A -- B
A 处理 B	A -- B
A 属于 B	A -- B
A 产生 B	A -- B
A 使用 B	A -- B
A 被 B 评价 / 投诉	A -- B

物业例子：

业主 发起 报修单
报修单 包含 调度记录
调度记录 包含 活动记录
维修工人 处理 调度记录
报修单 产生 评价单
报修单 产生 投诉单
活动记录 使用 维修材料

8.3 不要把实现关系画成领域关系

错误：

```
RepairService --> RepairRepository
RepairController --> RepairPage
RepairDatabase --> RepairOrder
```

正确：

```
业主 -- 报修单
报修单 -- 调度记录
调度记录 -- 活动记录
```

9. 多重性怎么判断

9.1 多重性定义

多重性表示：

一个 A 的实例能够和多少个 B 的实例关联。

常用写法：

多重性	含义
1	必须且只有一个
0..1	可以没有，最多一个
0..*	可以没有，也可以多个
1..*	至少一个，可以多个

9.2 多重性判断四问

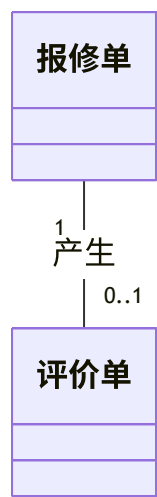
考试时逐个问：

- 1. 一个 A 最少对应几个 B？
- 2. 一个 A 最多对应几个 B？
- 3. 一个 B 最少对应几个 A？
- 4. 一个 B 最多对应几个 A？

例子：

一张报修单完成后可以有一张评价，但刚录入或处理中时还没有评价。

所以：



9.3 多重性要看生命周期

不要只看最终状态，要看对象生命周期中的任意合法状态。

例子：

关系	推荐多重性	理由
报修单 -- 评价单	1 -- 0..1	报修单刚创建时还没有评价
报修单 -- 投诉单	1 -- 0..*	可能没有投诉，也可能多次投诉
报修单 -- 费用单	1 -- 0..1	免费维修没有费用单
费用单 -- 支付记录	1 -- 0..1 或 0..*	未支付时没有记录；若允许分次支付则 0..*
调度记录 -- 活动记录	1 -- 0..*	刚派工时可能还没有活动记录
活动记录 -- 材料使用记录	1 -- 0..*	一次维修可能不用材料
巡检记录 -- 报修单	0..1 -- 0..1	不是所有巡检问题都会生成报修单

10. 关系有属性时，要抽中间类

这是领域模型题最高频得分点之一。

10.1 判断方法

如果你发现一个信息不单独属于 A，也不单独属于 B，而是属于：

A 和 B 的一次关系

那么要抽中间类。

10.2 典型例子一：活动记录和维修材料

需求：

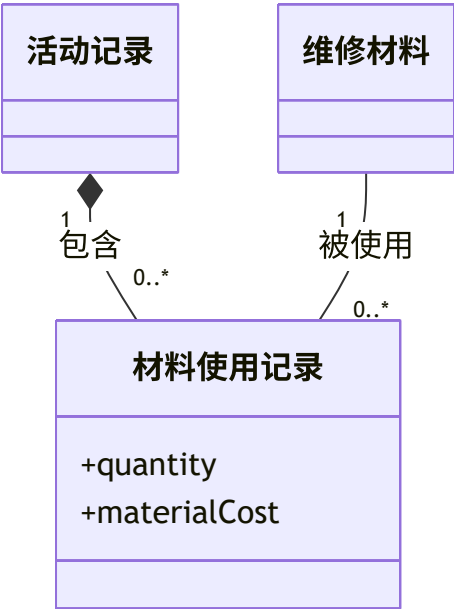
系统需要记录每次维修活动使用了哪些材料、使用数量和材料成本。

如果只画：

活动记录 -- 维修材料

不够，因为 `quantity` 和 `materialCost` 放哪里都不合适。

正确做法：



解释：

数量和成本属于“某次活动使用某种材料”这个关系，所以需要中间类“材料使用记录”。

10.3 典型例子二：调度记录和维修工人

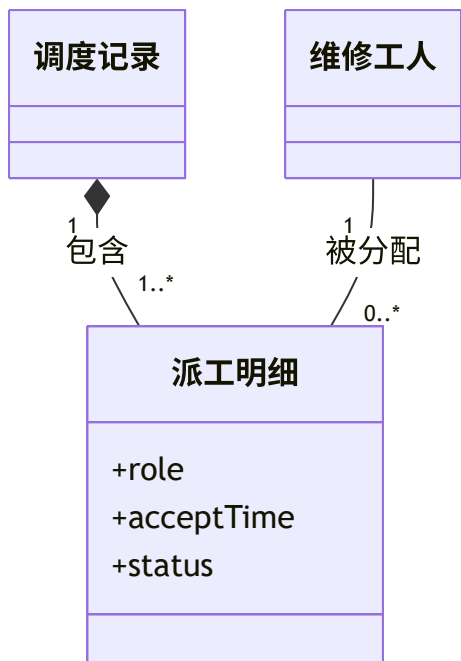
需求：

一个调度任务可以由多个维修工共同处理，系统记录每个人的角色、接收时间和完成状态。

错误：

调度记录 -- 维修工人

正确：



解释：

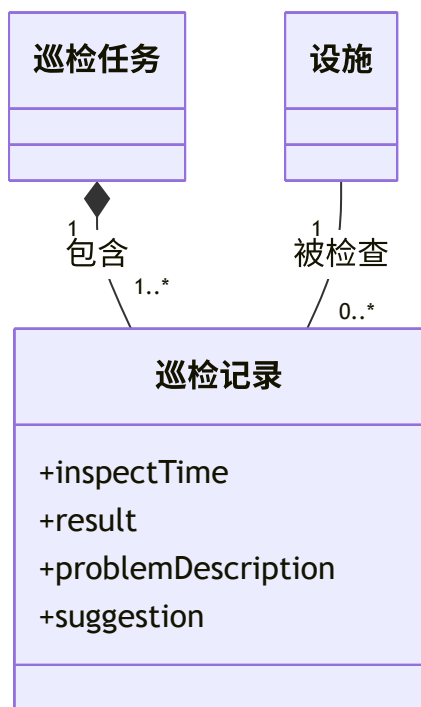
role、acceptTime、status 属于“某维修工参与某次调度”的关系，所以应建立“派工明细”。

10.4 典型例子三：巡检任务和设施

需求：

一个巡检任务包含多个设施，维修工需要为每个设施填写巡检时间、结果、问题描述和处理建议。

正确：



解释：

巡检结果不是单纯属于任务，也不是单纯属于设施，而是属于“某次任务检查某个设施”的结果，所以应建“巡检记录”。

11. 报表、票据、记录是否要建类

课件专门提醒要思考：

报表或票据是否是系统领域模型的元素？

答题时不要绝对化。

11.1 一般规则

如果它只是临时输出结果，不需要长期保存，一般不建类。

如果它需要保存、查询、追溯、参与业务流程，就应该建类。

11.2 例子

名词	是否建类	原因
查询结果	通常不建类	通常是临时输出
统计报表	看情况	如果要保存报表，则建类
小票 / Receipt	可以建类	如果它是交易记录、需要追溯
费用单	建类	有金额、状态、支付关系
支付记录	建类	有时间、金额、方式、状态
绩效考核表	建类	如果要保存考核结果
通知记录	看情况	只有需要保存通知状态时建类

考试答法：

报表或票据不一定是概念类。若它只是系统临时生成的展示结果，可以不建类；若它在业务中需要保存、查询、追溯，或者有自己的状态和关系，则应建成概念类。

12. 使用领域术语

领域模型应该尽量使用需求中的业务语言，而不是技术语言。

12.1 正确方向

使用：

报修单、调度记录、活动记录、维修工人、费用单、支付记录

不要优先使用：

RepairDTO、RepairVO、RepairTable、RepairController、SubmitForm

考试答法：

领域模型应使用领域术语表达业务概念，避免使用软件实现术语。

13. 组合、聚合、普通关联如何选择

考试中不一定强制区分，但如果要画得更清楚，可以这样处理。

13.1 组合 composition

适合表达强生命周期依赖：

报修单 *-- 调度记录

调度记录 *-- 活动记录

巡检任务 *-- 巡检记录

活动记录 *-- 材料使用记录

含义：

子对象通常依附于整体存在，是整体业务记录的一部分。

13.2 普通关联

适合表达相对独立的业务对象之间的关系：

维修工人 -- 调度记录

设施 -- 巡检记录

业主 -- 报修单

维修材料 -- 材料使用记录

13.3 考试简化策略

如果不确定组合还是普通关联：

先保证类、属性、多重性正确，关联类型可以保守画普通关联。
复杂约束用 note 说明。

14. 继承 / 泛化是否要画

领域模型可以画泛化关系，但不要滥用。

适合画继承的情况：

人员
↑
业主、调度员、维修工人、物业经理

前提是这些角色共享一些属性，例如 id、name、contact。

如果题目没有强调统一人员管理，也可以不画继承，直接分开画类。

考试建议：

继承不是必需项。能帮助简化共同属性时可以画；如果会增加复杂度，可以只画具体角色类。

15. 物业报修 Lab2 基础模型速查

下面是基于 Lab2 的基础领域对象，考试扩展时可以先从这里出发。

15.1 核心概念类

人员
业主
调度员
维修工人
物业经理

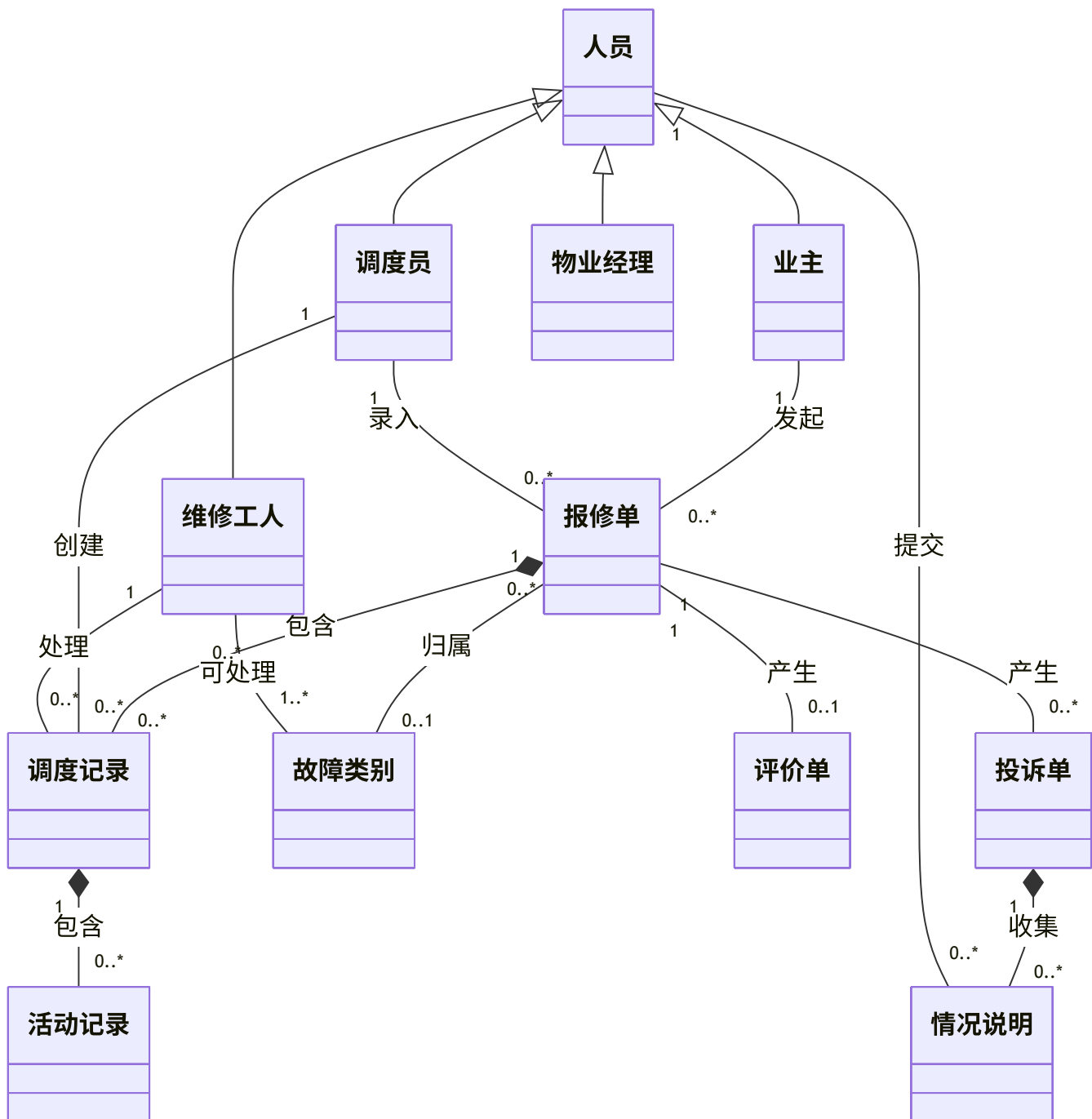
报修单
调度记录
活动记录
故障类别
评价单
投诉单
情况说明

15.2 核心属性

类	常见属性
人员	id、name、contact

类	常见属性
业主	ownerId、contact
调度员	employeeId、name
维修工人	employeeId、name、/isIdle
物业经理	employeeId、name
报修单	repairId、requestTime、faultDescription、source、status
调度记录	dispatchId、dispatchTime、finishTime、status、remark
活动记录	activityId、startTime、endTime、processDetail
故障类别	categoryId、categoryName
评价单	evaluateTime、responseTimeliness、serviceAttitude、satisfactionResult
投诉单	complaintTime、content、status
情况说明	content、submitTime、status

15.3 核心关系



15.4 重要约束 note

- 约束 1：一个报修单可以多次调度，但同一时刻最多只有一个活动中的调度记录。
- 约束 2：情况说明的提交者应是参与该报修处理的调度员或维修工人。
- 约束 3：报修单完成后可以产生评价单，但处理中时可以没有评价单。
- 约束 4：投诉单可以在评价后产生，也可以根据题目需求从报修单产生。

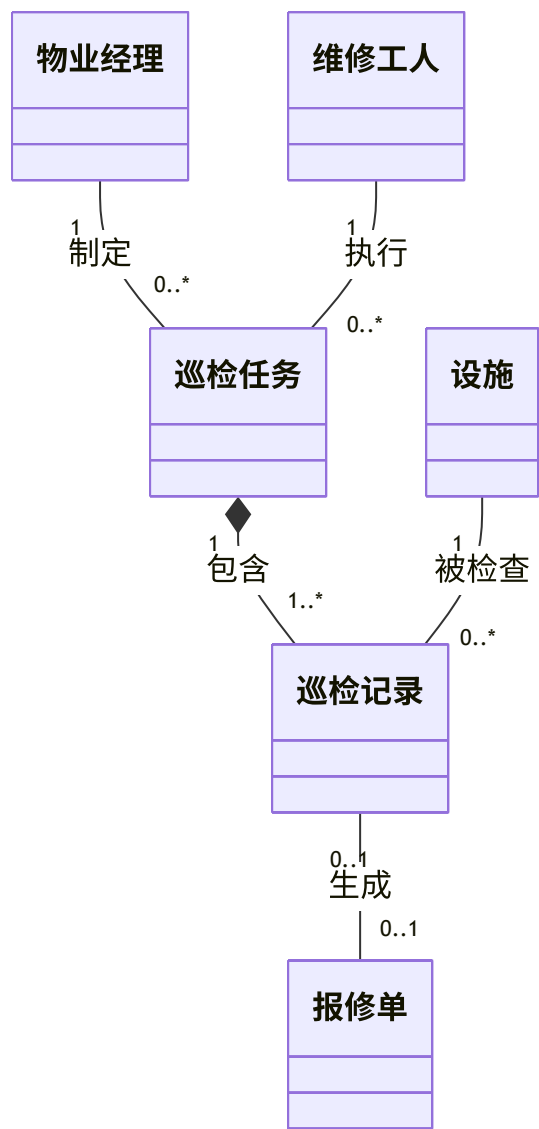
16. 常见扩展方向速查

16.1 日常巡检

新增：

巡检任务
巡检记录
设施

关系：

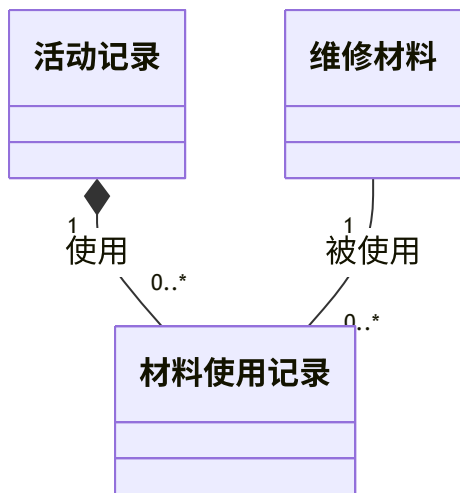


16.2 维修材料和成本统计

新增：

维修材料
材料使用记录

关系：



关键解释：

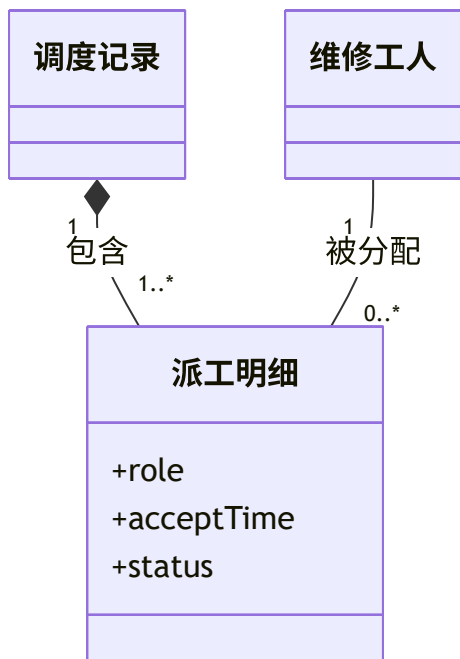
材料使用记录是中间类，因为 quantity、materialCost 属于“某次活动使用某种材料”这个关系。

16.3 多人协作维修

新增：

派工明细

关系：

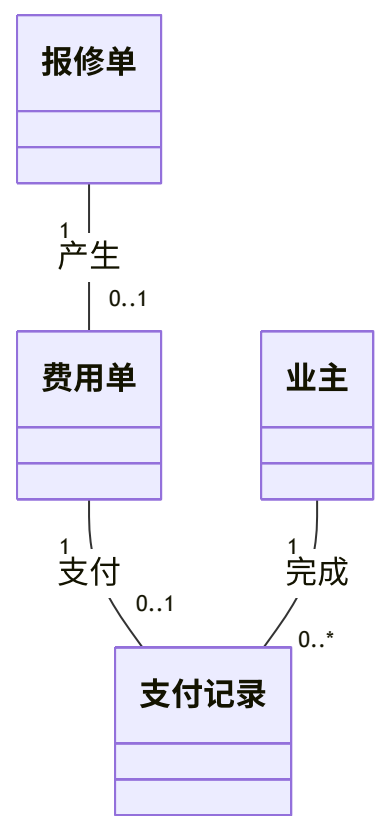


16.4 费用结算

新增：

费用单
支付记录

关系：



注意：

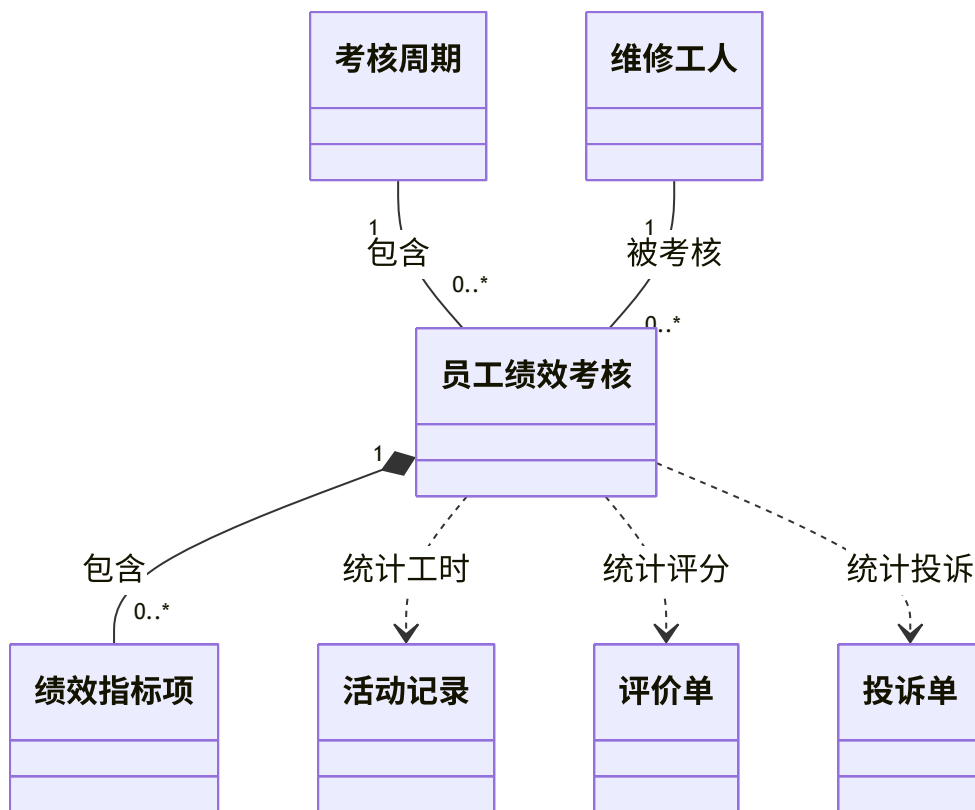
如果题目说允许分次支付，费用单到支付记录改成 0..*。

16.5 员工业绩考核

新增：

考核周期
员工绩效考核
绩效指标项

关系：



16.6 排班 / 空闲判断

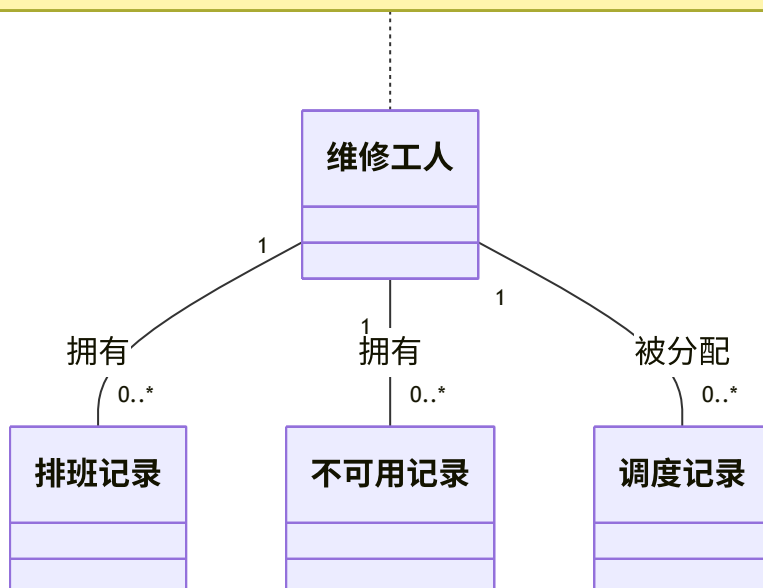
新增:

排班记录

不可用记录

关系:

是否空闲通常由排班记录、不可用记录、活动中的调度记录共同推导



17. 领域模型答题模板

17.1 文字说明模板

可以直接套：

本领域模型主要包含以下概念类：.....

这些概念类都是需求中需要长期保存、查询或统计的业务对象。

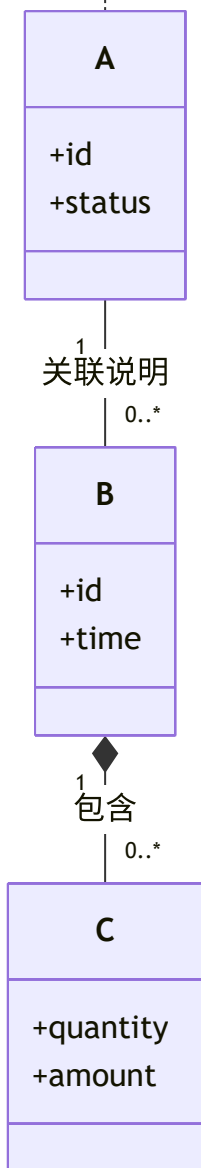
对于关系本身具有属性的情况，例如.....，将其提升为中间概念类.....。

多重性根据对象生命周期判断，例如.....在刚创建时可以没有.....，因此多重性为 0..1 或 0..*。

本模型只表达业务概念、属性和关联，不包含页面、按钮、数据库表、控制器、算法或具体实现方法。

17.2 Mermaid 答题骨架

复杂业务约束写在 note 中



18. 高频判断题

1. 领域模型是对领域内概念类的可视化表示。
对。
2. 领域模型就是数据库表设计。
错。领域模型关注领域概念，数据模型关注持久化。
3. 领域模型中应该包含窗口、按钮、控制器和数据库表。
错。领域模型不是软件对象模型。
4. 领域模型中通常不定义操作或方法。
对。领域模型提供概念透视图，主要展示概念类、属性、关联。
5. 需求中出现的所有名词都应该建成概念类。
错。名词只是候选项，需要筛选。

6. 如果两个类之间的关系本身有属性，应考虑建立中间类。

对。

7. 多重性只需要看最终状态，不需要看对象生命周期。

错。生命周期中的合法状态也要考虑。

8. 报修单刚录入时没有评价，因此报修单到评价单可以是 0..1。

对。

9. “通知报修人”一定要建通知类。

错。只有需要保存通知记录时才建类。

10. 领域模型应使用领域术语，而不是技术实现术语。

对。

19. 最短口诀

领域模型三件事：概念类、属性、关联。

找类：

圈名词，筛对象；

要保存、要查询、要统计、有属性、有关系，就建类。

加属性：

编号、时间、状态、内容、金额、数量、评分。

建关系：

发起、录入、创建、包含、处理、产生、使用、属于。

定多重性：

问 A 对 B 最少最多，B 对 A 最少最多；

看生命周期，不只看最终状态。

遇到关系属性：

抽中间类。

易错：

不画页面按钮数据库；

不画方法算法流程；

不把所有名词都建类；

复杂约束写 note。